IT STUDY HUB

COMPLETE C++ PROGRAMMING SERIES

MODULE 11: DATA STRUCTURES & ALGORITHMS WITH C++

Authors: Mithila Rani Saha & Dithsa Dutta

Website: www.itstudyhub.dpdns.org

Module Overview

1. About This Module

Welcome to **Module 11: Data Structures & Algorithms with C++** in the IT Study Hub Complete C++ Programming Series. Building upon the concurrency and networking mastery of Module 10, this module dives deep into computational complexity, foundational data structures, advanced tree/graph topologies, sorting/searching paradigms, and algorithm design techniques

(Greedy, Dynamic Programming, Backtracking, Divide and Conquer). Authored by Mithila Rani Saha and Dithsa Dutta, this handbook implements critical data structures and algorithms from scratch in C++.

2. Learning Outcomes

By completing this module, you will be able to:

- Analyze algorithm efficiency using Big O notation, time complexity, space complexity, and amortized analysis.
- Implement foundational linear data structures (Arrays, Singly/Doubly/Circular Linked Lists, Stacks, Queues, Deques) from scratch.
- Master hierarchical and associative data structures (Binary Search Trees, AVL Trees, Heaps, Hash Tables, Graphs).
- Design and implement searching, sorting, graph traversal (BFS, DFS), and paradigm algorithms (Greedy, DP, Backtracking).
- Architect a complete, reusable Data Structures & Algorithms C++ library.

3. Module Coverage

This module comprehensively covers:

- **Algorithm Analysis:** Big O notation, time/space complexity, best/average/worst cases, and amortized analysis.
- **Linear Data Structures:** Arrays, Linked Lists (Singly, Doubly, Circular), Stacks, Queues, and Deques.
- **Advanced Data Structures:** Hash Tables, Binary Trees, BSTs, AVL Trees, Heaps, and Graphs (Adjacency Matrix/List).
- **Algorithms & Paradigms:** BFS, DFS, Sorting (Bubble, Selection, Insertion, Merge, Quick, Heap), Searching (Linear, Binary), Greedy, Dynamic Programming, Backtracking, and Divide & Conquer.

- **Hands-On Project:** Complete Data Structures & Algorithms C++ Library.
- **Rigorous Assessments:** Big O Cheat Sheet, 20+ practice questions, 25+ viva questions, 25+ DSA interview questions, and multi-level coding challenges.

IT STUDY HUB

4. Main Notes: Theoretical Foundations

Data Structures and Algorithms form the computational bedrock of efficient software engineering. Below are the exhaustive foundational notes for Module 11.

Concept: Algorithm Analysis & Big O Notation

Simple Explanation: Algorithm analysis evaluates the computational resources (time and memory) required as input size N grows. Big O notation describes the asymptotic upper bound of growth rates.

> ***Real-Life Analogy:*** *Searching for a name in a physical phone book by flipping pages one by one is like $O(N)$ linear search. Opening the phone book precisely in the middle repeatedly to narrow down the page is like $O(\log N)$ binary search.*

> **Structure / Diagram:**
> **Input Size N vs Execution Time / Operations Growth Curve**

```
// Big O Growth Rate Hierarchy:
// O(1) < O(log n) < O(n) < O(n log n) < O(n^2) < O(2^n) < O(n!)
```

> **Complexity Summary:**
> Best, Average, and Worst cases quantify performance across ideal, random, and adversarial inputs.

Real-World Application: Evaluating database query scalability and sorting massive enterprise datasets.

Common Mistakes: Confusing best-case performance with typical real-world average performance.

Best Practices: Always analyze worst-case time and space complexity when designing scalable backend architectures.

Actionable Practice: Determine the Big O time complexity of a nested loop running N times inside another loop running N times.

Concept: Singly Linked List Implementation from Scratch

Simple Explanation: A singly linked list is a linear data structure where elements are stored in nodes. Each node holds data and a pointer to the next node in the sequence.

> ***Real-Life Analogy:*** *A scavenger hunt where each location holds a clue and the precise address pointing to the next exact location.*

Structure / Diagram:

```
[ Head ] -> [ Data | Next* ] -> [ Data | Next* ] -> nullptr
```

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class SinglyLinkedList {
```

```
    private:
        Node* head;
    public:
        SinglyLinkedList() : head(nullptr) {}

        void insertFront(int val) {
            Node* newNode = new Node(val);
            newNode->next = head;
            head = newNode;
        }

        void display() {
            Node* temp = head;
            while(temp != nullptr) {
                cout << temp->data << " -> ";
                temp = temp->next;
            }
            cout << "nullptr
";
        }

        ~SinglyLinkedList() {
            Node* current = head;
            while(current != nullptr) {
                Node* nextNode = current->next;
                delete current;
                current = nextNode;
            }
        }
};

int main() {
    SinglyLinkedList list;
    list.insertFront(30);
    list.insertFront(20);
    list.insertFront(10);
    list.display();
    return 0;
}
```

Output:

```
10 -> 20 -> 30 -> nullptr
```

Line-by-Line Breakdown:

- **struct Node**: Defines the linked list node containing data and a pointer to the next node.
- **insertFront()**: Creates a new node and prepends it to the head pointer in $O(1)$ time.
- **display()**: Traverses the list sequentially from head to tail.

Real-World Application: Implementing dynamic memory pools, undo stacks, and symbol tables.

> **Common Mistakes:** Losing the head pointer reference during node insertion or deletion, causing memory leaks and broken links.

> **Best Practices:** Always clean up dynamically allocated node memory in destructors to prevent memory leaks.

> **Actionable Practice:** Add an `append` method to insert nodes at the tail of a singly linked list.

Concept: Binary Search Tree (BST) & Traversals

Simple Explanation: A Binary Search Tree is a hierarchical tree where each node has at most two children, and the left child's key is less than the parent's key while the right child's key is greater.

> ***Real-Life Analogy:*** *A corporate organizational chart or a family tree sorted numerically by ID numbers.*

> **Structure / Diagram:**
> **Root (50) → Left child (30) < 50 < Right child (70)**

```
#include <iostream>
using namespace std;
```

```
struct BSTNode {
    int data;
    BSTNode* left;
    BSTNode* right;
    BSTNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

class BST {
    private:
        BSTNode* insert(BSTNode* node, int val) {
            if(!node) return new BSTNode(val);
            if(val < node->data) node->left = insert(node->left,
val);
            else if(val > node->data) node->right = insert(node-
>right, val);
            return node;
        }

        void inorder(BSTNode* root) {
            if(!root) return;
            inorder(root->left);
            cout << root->data << " ";
            inorder(root->right);
        }
    public:
        BSTNode* root = nullptr;
        void insert(int val) { root = insert(root, val); }
        void displayInorder() { inorder(root); cout << endl; }
};

int main() {
    BST tree;
    tree.insert(50);
    tree.insert(30);
    tree.insert(70);
    tree.insert(20);
    tree.insert(40);

    cout << "Inorder Traversal (Sorted): ";
    tree.displayInorder();
    return 0;
}
```

```
Output:
Inorder Traversal (Sorted): 20 30 40 50 70
```

Line-by-Line Breakdown:

- **insert()**: Recursively compares values to route nodes to left or right subtrees.
- **inorder()**: Recursively visits left subtree, root, and right subtree to produce sorted output.

Real-World Application: Database indexing and associative map implementations.

Common Mistakes: Inserting sorted data into a BST without balancing, degrading tree height to $O(N)$.

Best Practices: Use self-balancing trees like AVL Trees or Red-Black Trees for guaranteed $O(\log N)$ operations.

Actionable Practice: Implement a search function in the BST class returning `true` if a value exists.

Big O Cheat Sheet

Quick reference guide for algorithm time and space complexities across standard data structures and algorithms.

Complexity Class	Big O Notation	Example Algorithm / Operation
Constant Time	$O(1)$	Array index access, Hash table lookup (average), Stack push/pop
Logarithmic Time	$O(\log N)$	Binary search, AVL tree operations, Heap insert/extract
Linear Time	$O(N)$	Linear search, Array traversal, Singly linked list traversal
Linearithmic Time	$O(N \log N)$	Merge sort, Quick sort, Heap sort
Quadratic Time	$O(N^2)$	Bubble sort, Selection sort, Insertion sort, Nested matrix loops
Exponential Time	$O(2^N)$	Recursive Fibonacci, Subset generation (Power Set)
Factorial Time	$O(N!)$	Traveling Salesperson Problem (Brute Force permutations)

Comparative Analysis

Professional C++ software engineering requires mastering trade-offs between linear, tree, graph, and sorting algorithms.

Table 1: Array vs Linked List

Feature	Array	Linked List
Memory Layout	Contiguous RAM blocks.	Scattered nodes linked via pointers.
Access Time	$O(1)$ random access.	$O(N)$ sequential traversal.
Insertion/ Deletion	$O(N)$ due to element shifting.	$O(1)$ pointer adjustment if position is known.

Table 2: Merge Sort vs Quick Sort vs Heap Sort

Algorithm	Time Complexity (Average)	Time Complexity (Worst)	Space Complexity	Stable?
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N)$	Yes
Quick Sort	$O(N \log N)$	$O(N^2)$	$O(\log N)$	No
Heap Sort	$O(N \log N)$	$O(N \log N)$	$O(1)$	No

HANDS-ON PROJECT: COMPLETE DATA STRUCTURES & ALGORITHMS LIBRARY

In this comprehensive hands-on project, we assemble a modular C++ library uniting dynamic arrays, linked lists, binary search trees, sorting algorithms, and searching utilities.

Project Specification

Problem Statement: Build a reusable Data Structures & Algorithms C++ library uniting custom linear structures, tree nodes, sorting routines, and searching algorithms into a unified test harness.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// Library Component: Binary Search Utility
int binarySearch(const vector<int> &arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

int main() {
    cout << "========================================" << endl;
    cout << " IT STUDY HUB DSA LIBRARY TEST SUITE    " << endl;
    cout << "========================================" << endl;

    vector<int> dataset = {10, 23, 45, 70, 85, 90, 110};
    int target = 85;
```

```
    int resultIndex = binarySearch(dataset, target);

    if (resultIndex != -1) {
        cout << "Target " << target << " found at index: " <<
resultIndex << endl;
    } else {
        cout << "Target not found in dataset." << endl;
    }

    cout << "\nDSA library test completed successfully." << endl;
    return 0;
}
```

Expected Output:

```
========================================
 IT STUDY HUB DSA LIBRARY TEST SUITE
========================================
Target 85 found at index: 4

DSA library test completed successfully.
```

Step-by-Step Explanation:

- `binarySearch` implements $O(\log N)$ divide-and-conquer searching on sorted vectors.
- Computes middle index safely to prevent integer overflow.
- Returns target index or -1 if absent.

Practice Challenges:

- Implement Merge Sort from scratch and integrate it into the DSA library.
- Implement Breadth-First Search (BFS) for graph traversal.

Module Assessments

Test your mastery of Data Structures & Algorithms with these rigorous assessment questions.

20+ Practice Questions

1. What is Big O notation, and why is it used?
2. Compare arrays and linked lists in terms of memory and access time.
3. Implement a singly linked list reversal function.
4. What is the difference between a stack and a queue?
5. Explain how hash collisions occur and how chaining resolves them.
6. What is a Binary Search Tree (BST), and what is its worst-case search complexity?
7. How do AVL trees maintain balance during insertions?
8. Compare Min Heap and Max Heap structures.
9. Explain BFS versus DFS traversal algorithms.
10. What is the time complexity of Quick Sort in the worst case?
11. Implement binary search iteratively in C++.
12. Explain the Greedy algorithm paradigm with an example.
13. What is Dynamic Programming, and how does memoization optimize recursive solutions?
14. Explain Backtracking using the N-Queens problem.
15. What is Divide and Conquer? Give an example.
16. Implement bubble sort from scratch.
17. What is an adjacency list representation for graphs?
18. Explain amortized analysis with respect to `std::vector` capacity doubling.
19. What is the difference between Dijkstra's algorithm and Breadth-First Search?
20. Implement a queue using two stacks.

25+ Viva Questions (With Answers)

Q1: What is an algorithm?

> A step-by-step set of unambiguous instructions designed to solve a specific computational problem.

Q2: What is time complexity?

> A theoretical quantification of the amount of execution time taken by an algorithm as a function of input size.

Q3: What is space complexity?

> The total amount of memory space required by an algorithm relative to input size.

Q4: What is a linked list?

> A linear data structure consisting of nodes where each node contains data and a reference to the next node.

Q5: What is a stack?

> A LIFO (Last-In-First-Out) linear data structure.

Q6: What is a queue?

> A FIFO (First-In-First-Out) linear data structure.

Q7: What is a hash table?

> A data structure that maps keys to values using a hash function for efficient $O(1)$ average lookups.

Q8: What is a binary tree?

> A hierarchical tree data structure in which each node has at most two children.

Q9: What is an AVL tree?

> A self-balancing binary search tree where heights of left and right subtrees differ by at most one.

Q10: What is a heap?

> A specialized tree-based complete binary tree satisfying the heap property.

Q11: What is a graph?

> A non-linear data structure consisting of vertices (nodes) connected by edges.

Q12: What is BFS?

> Breadth-First Search—a graph traversal algorithm exploring neighbor nodes level by level using a queue.

Q13: What is DFS?

> Depth-First Search—a graph traversal algorithm exploring as deep as possible along each branch using recursion or a stack.

Q14: What is Merge Sort?

> A divide-and-conquer sorting algorithm dividing arrays into halves, sorting recursively, and merging them in $O(N \log N)$ time.

Q15: What is Quick Sort?

> A divide-and-conquer sorting algorithm partitioning arrays around a pivot element.

Q16: What is binary search?

> A logarithmic search algorithm operating on sorted arrays by repeatedly dividing the search interval in half.

Q17: What is a greedy algorithm?

> An algorithmic paradigm making the locally optimal choice at each stage with the intent of finding a global optimum.

Q18: What is dynamic programming?

> An optimization technique breaking complex problems down into simpler overlapping subproblems and storing results.

Q19: What is backtracking?

> An algorithmic technique building candidate solutions incrementally and abandoning ("backtracking") branches that fail constraints.

Q20: What is amortized analysis?

> A method of analyzing the average cost per operation over a sequence of operations.

Q21: What is an adjacency matrix?

> A 2D array representing graph vertices where cells indicate edge connections.

Q22: What is an adjacency list?

> An array of linked lists representing graph vertices and their adjacent neighbors.

Q23: What is a stable sorting algorithm?

> An algorithm that preserves the relative order of identical elements in the sorted output.

Q24: What is memoization?

> Storing the results of expensive function calls to cache and reuse in dynamic programming.

Q25: Why is hash table collision resolution necessary?

> Because multiple unique keys can hash to the same bucket index.

25+ DSA Interview Questions

1. **How do you detect a cycle in a singly linked list?** (Floyd's Cycle-Finding Algorithm using slow and fast pointers).
2. **How do you reverse a linked list iteratively and recursively?**
3. **Implement an LRU (Least Recently Used) Cache using a hash map and doubly linked list.**
4. **How do you find the lowest common ancestor (LCA) in a Binary Search Tree?**
5. **Implement Level Order Traversal (BFS) on a binary tree.**
6. **How do you find the median of a running stream of integers?** (Using two heaps: max-heap and min-heap).
7. **Explain the Knapsack Problem (0/1 vs Fractional) and solve it using Dynamic Programming.**
8. **How do you find the shortest path in a weighted graph?** (Dijkstra's Algorithm).
9. **Detect a cycle in a directed graph using DFS.**
10. **How do you find connected components in an undirected graph?**
11. **Implement Merge Sort and explain its divide-and-conquer mechanics.**

12. **Find the Kth largest element in an unsorted array using a min-heap.**
13. **Check if a binary tree is a valid Binary Search Tree (BST).**
14. **Implement Trie (Prefix Tree) insert, search, and startsWith operations.**
15. **Find the maximum subarray sum (Kadane's Algorithm).**
16. **Implement an in-place array rotation algorithm.**
17. **Find the first non-repeating character in a string using hashing.**
18. **Solve the Coin Change problem using dynamic programming.**
19. **Implement Depth-First Search (DFS) for graph traversal.**
20. **How do you balance an unbalanced BST? (AVL Rotations: LL, RR, LR, RL).**
21. **Find the length of the Longest Increasing Subsequence (LIS).**
22. **Implement a Stack supporting `getMin()` in $O(1)$ time.**
23. **Find the top K frequent elements in an array.**
24. **Solve the Rat in a Maze problem using backtracking.**
25. **Explain topological sorting in directed acyclic graphs (DAGs).**

IT STUDY

Module 11 Wrap-Up

Key Terms Glossary

- **Big O Notation:** Asymptotic upper bound measuring algorithm scalability.
- **Linked List:** Linear nodes connected via pointers.
- **BST:** Binary Search Tree with ordered left/right children.
- **AVL Tree:** Self-balancing binary search tree.
- **Heap:** Complete binary tree satisfying heap ordering property.
- **BFS / DFS:** Breadth-First and Depth-First graph traversals.
- **Dynamic Programming:** Optimization solving overlapping subproblems via memoization/ tabulation.

Quick Revision Summary

Module 11 establishes data structures and algorithms. Complexity analysis uses Big O notation to evaluate time and space scalability. Linear structures include arrays, linked lists, stacks, queues, and deques. Advanced structures encompass hash tables, trees, AVL trees, heaps, and graphs. Algorithms span searching, sorting, graph traversals, greedy methods, dynamic programming, and backtracking. Mastering DSA empowers engineers to write optimal, scalable software systems.

Final DSA Cheat Sheet & Coding Challenges

Complete these final challenges before moving to Module 12:

- **Challenge 1 (Beginner):** Implement a stack using linked lists.
- **Challenge 2 (Intermediate):** Find the intersection node of two singly linked lists.
- **Challenge 3 (Advanced):** Implement Dijkstra's shortest path algorithm using an adjacency list and priority queue.
- **Challenge 4 (Competitive):** Solve the Edit Distance problem using 2D dynamic programming.

CONGRATULATIONS ON COMPLETING MODULE 11!

WHAT COMES NEXT:

MODULE 12 — ADVANCED C++, SOFTWARE ENGINEERING & PRODUCTION